Jupyter Notebook 编程面试出题形式指南

PyTorch Computer Vision Interview - Notebook Structure, TODO Patterns, and Response Workflow

这份文档整理了面试中以 Jupyter Notebook 出题时常见的内容格式、cell 结构、TODO 形式，以及你应该如何按步骤完成 PyTorch 计算机视觉任务。重点覆盖图像分类、Transformer bonus、目标检测任务，以及 notebook 中常见报错和应对流程。

1. Jupyter Notebook 的基本内容格式

Jupyter Notebook 本质上是一个 .ipynb 文件，内容由多个 cell 顺序组成。面试官通常会把任务拆成文字说明和代码空白，让你一步一步完成。

```
Notebook Title
->
Markdown Cell: task description
->
Code Cell: import packages
->
Markdown Cell: dataset description
->
Code Cell: load/check dataset
->
Markdown Cell: TODO 1 - data transforms
->
Code Cell: implement transforms and dataloaders
->
Markdown Cell: TODO 2 - model
->
Code Cell: build model
->
Markdown Cell: TODO 3 - training loop
->
Code Cell: train and validate
->
Markdown Cell: TODO 4 - results/explanation
->
Code Cell: plot curves and visualize predictions
```

两种主要 cell

- Markdown Cell: 用于题目说明、任务要求、数据结构说明、问题提示，不运行代码。
- Code Cell: 用于 Python 代码执行，通常会包含 import、数据加载、模型定义、训练和评估。

2. 一个典型分类任务 Notebook 会长什么样

图像分类任务 notebook 通常会按下面的顺序组织。你应该先读任务，再检查数据，最后补全模型和训练代码。

Section 1 - Task description

```
# Image Classification Task

You are given an image dataset in the following format:

Data/
    train/
        cats/
        dogs/
    val/
        cats/
        dogs/
```

```
Build a PyTorch model to classify each image as cat or dog.
```

看到这种结构，你应该马上判断：这是 image classification，不是 object detection。

Section 2 - Imports

```
import os
import torch
import torch.nn as nn
import torch.optim as optim
from torchvision import datasets, transforms, models
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

Section 3 - Dataset path

```
DATA_DIR = "Data"

print(os.getcwd())
print(os.listdir("."))
print(os.listdir(DATA_DIR))
print(os.listdir(os.path.join(DATA_DIR, "train")))
print(os.listdir(os.path.join(DATA_DIR, "val")))
```

面试时不要马上写模型。先检查路径和数据结构，这会显得你很稳。

Section 4 - Data transforms

```
data_transforms = {
    "train": transforms.Compose([
        transforms.Resize((64, 64)),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ]),
    "val": transforms.Compose([
        transforms.Resize((64, 64)),
        transforms.ToTensor(),
        transforms.Normalize(
            mean=[0.485, 0.456, 0.406],
            std=[0.229, 0.224, 0.225]
        )
    ])
}
```

如果使用 ResNet transfer learning，通常用 224 x 224；如果使用 CPU-friendly MiniViT，可以用 64 x 64。

Section 5 - Dataset and DataLoader

```
image_datasets = {
    phase: datasets.ImageFolder(
        root=os.path.join(DATA_DIR, phase),
        transform=data_transforms[phase]
    )
    for phase in ["train", "val"]
}

dataloaders = {
    phase: DataLoader(
        image_datasets[phase],
        batch_size=4,
        shuffle=True if phase == "train" else False,
        num_workers=0
```

```
    )
    for phase in ["train", "val"]
}

dataset_sizes = {
    phase: len(image_datasets[phase])
    for phase in ["train", "val"]
}

class_names = image_datasets["train"].classes
print(class_names)
print(dataset_sizes)
```

Section 6 - Visualize sample images

```
inputs, labels = next(iter(dataloaders["train"]))
print(inputs.shape)
print(labels)

# Optional: display a batch
import torchvision
out = torchvision.utils.make_grid(inputs)
plt.imshow(out.permute(1, 2, 0))
plt.show()
```

如果图像经过 Normalize，直接显示可能颜色怪。正式展示时建议写 unnormalize 函数。

3. Model Cell 的几种常见形式

ResNet transfer learning 版本

```
model = models.resnet18(weights="IMAGENET1K_V1")
num_ftrs = model.fc.in_features
model.fc = nn.Linear(num_ftrs, len(class_names))
model = model.to(device)
```

这是最稳定、最适合快速拿到结果的分类 baseline。

MiniViT Transformer bonus 版本

```
class PatchEmbedding(nn.Module):
    def __init__(self, img_size, patch_size, in_channels, d_model):
        super().__init__()
        self.num_patches = (img_size // patch_size) ** 2
        self.proj = nn.Conv2d(
            in_channels, d_model,
            kernel_size=patch_size,
            stride=patch_size
        )

    def forward(self, x):
        x = self.proj(x)       # [B, D, H/P, W/P]
        x = x.flatten(2)       # [B, D, N]
        x = x.transpose(1, 2)  # [B, N, D]
        return x

class MiniViT(nn.Module):
    def __init__(self, num_classes):
        super().__init__()
        self.patch_embed = PatchEmbedding(64, 8, 3, 64)
        self.cls_token = nn.Parameter(torch.zeros(1, 1, 64))
        self.pos_embed = nn.Parameter(torch.zeros(1, 65, 64))
        encoder_layer = nn.TransformerEncoderLayer(
            d_model=64, nhead=4,
            dim_feedforward=128,
            batch_first=True
```

```
        )
        self.encoder = nn.TransformerEncoder(encoder_layer, num_layers=1)
        self.norm = nn.LayerNorm(64)
        self.head = nn.Linear(64, num_classes)

    def forward(self, x):
        b = x.shape[0]
        x = self.patch_embed(x)
        cls = self.cls_token.expand(b, -1, -1)
        x = torch.cat([cls, x], dim=1)
        x = x + self.pos_embed
        x = self.encoder(x)
        x = self.norm(x[:, 0])
        return self.head(x)

model = MiniViT(num_classes=len(class_names)).to(device)
```

如果 notebook 的 bonus 是 Transformer 分类，可以使用这种 MiniViT。它适合 CPU 演示，但准确率通常不如 pretrained ResNet 稳。

4. Loss, Optimizer, Scheduler Cell

分类任务通常使用 CrossEntropyLoss。ResNet 可以用 SGD；Transformer 更常用 AdamW。

```
criterion = nn.CrossEntropyLoss()
optimizer = optim.AdamW(model.parameters(), lr=1e-3, weight_decay=1e-4)
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=2, gamma=0.5)
```

5. Training Loop Cell

训练 cell 通常要求你完成 forward、loss、backward、optimizer.step，以及统计 loss 和 accuracy。

```
num_epochs = 3
history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}

for epoch in range(num_epochs):
    model.train()
    running_loss = 0.0
    running_corrects = 0
    total = 0

    for inputs, labels in dataloaders["train"]:
        inputs = inputs.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)

        loss.backward()
        optimizer.step()

        running_loss += loss.item() * inputs.size(0)
        running_corrects += torch.sum(preds == labels).item()
        total += inputs.size(0)

    train_loss = running_loss / total
    train_acc = running_corrects / total

    print(f"Epoch {epoch+1}, Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f}")
```

6. Validation Loop Cell

验证阶段必须使用 model.eval() 和 torch.no_grad()，不要反向传播。

```
model.eval()
val_loss = 0.0
val_corrects = 0
val_total = 0

with torch.no_grad():
    for inputs, labels in dataloaders["val"]:
        inputs = inputs.to(device)
        labels = labels.to(device)

        outputs = model(inputs)
        loss = criterion(outputs, labels)
        _, preds = torch.max(outputs, 1)

        val_loss += loss.item() * inputs.size(0)
        val_corrects += torch.sum(preds == labels).item()
        val_total += inputs.size(0)

val_loss = val_loss / val_total
val_acc = val_corrects / val_total
print(f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")
```

7. Plot Curves and Visualize Predictions

Plot loss/accuracy curves

```
plt.plot(history["train_loss"], label="Train Loss")
plt.plot(history["val_loss"], label="Val Loss")
plt.legend()
plt.show()

plt.plot(history["train_acc"], label="Train Acc")
plt.plot(history["val_acc"], label="Val Acc")
plt.legend()
plt.show()
```

Visualize predictions

```
model.eval()
inputs, labels = next(iter(dataloaders["val"]))
inputs = inputs.to(device)

with torch.no_grad():
    outputs = model(inputs)
    _, preds = torch.max(outputs, 1)

for i in range(min(4, inputs.size(0))):
    img = inputs[i].cpu()
    plt.figure()
    plt.imshow(img.permute(1, 2, 0))
    plt.title(f"Pred: {class_names[preds[i]]}, True: {class_names[labels[i]]}")
    plt.axis("off")
    plt.show()
```

如果图像 normalize 过，需要先反归一化再显示。

8. Object Detection Notebook 和分类 Notebook 的区别

如果题目是目标检测，notebook 会要求返回 image 和 target，而不是 image 和 label。target 至少包含 boxes 和 labels。

```
target = {
    "boxes": boxes,      # FloatTensor[N, 4], format: [xmin, ymin, xmax, ymax]
    "labels": labels     # Int64Tensor[N]
}
```

检测任务通常需要自定义 Dataset，并且 DataLoader 要使用 collate_fn，因为每张图的目标数量不同。

```
def collate_fn(batch):
    return tuple(zip(*batch))
```

9. Notebook 中常见 TODO 形式

面试 notebook 中常见的空白形式包括：

- TODO: Fill in this function
- START CODE HERE / END CODE HERE
- raise NotImplementedError
- Complete the model definition
- Implement the training loop
- Report validation accuracy

```
def build_model(num_classes):
    # TODO: implement model here
    raise NotImplementedError

### START CODE HERE ###
# your code
### END CODE HERE ###
```

10. Notebook 运行顺序和常见错误

Jupyter Notebook 不像 .py 文件总是从头到尾运行。你修改了某个 cell 后，必须重新运行依赖它的后续 cell。

- 修改 import 或 config 后，要重新运行后面的 cell。
- 修改 class definition 后，要重新运行 model creation cell。
- 修改 DATA_DIR 后，要重新运行 dataloader cell。
- 修改 model structure 后，要重新创建 optimizer。
- 如果出现 NameError，通常是上面的 cell 没运行。
- 如果出现 FileNotFoundError，通常是 DATA_DIR 或文件夹结构不对。
- 如果出现 shape mismatch，通常是模型输出维度、类别数或输入 shape 不对。

11. 面试拿到 Notebook 后的推荐动作

建议按照下面的顺序操作，不要直接开始写模型：

```
import os

print(os.getcwd())
print(os.listdir("."))
print(os.listdir("Data"))
print(os.listdir("Data/train"))
print(os.listdir("Data/val"))

for cls in os.listdir("Data/train"):
    cls_path = os.path.join("Data", "train", cls)
    print(cls, len(os.listdir(cls_path)))
```

12. 面试解释话术

你可以这样解释 notebook 工作流程：

The notebook is usually organized into markdown cells and code cells. I will first read the task description and inspect the dataset structure. Then I will define the transforms, build the dataset and dataloaders, define the model, loss function, optimizer, and training loop. After training, I will evaluate validation accuracy, visualize predictions, and explain the design choices. If the notebook includes TODO sections, I will fill them step by step and run each cell to verify the output.

中文理解：Jupyter Notebook 通常由 markdown cell 和 code cell 组成。我会先阅读任务描述并检查数据结构。然后定义图像预处理、构建 dataset 和 dataloader、定义模型、损失函数、优化器和训练循环。训练后，我会评估验证准确率、可视化预测结果，并解释设计选择。如果 notebook 中有 TODO 区域，我会逐步补全并运行每个 cell 来确认输出正确。

13. 最后总结

Notebook 出题形式通常是：Markdown 说明 + Code Cell 代码 + TODO 空白。你需要熟悉下面这个顺序：

- 读任务
- 检查数据
- 写 transforms
- 写 dataset/dataloader
- 写 model
- 写 loss/optimizer
- 写 training loop
- 写 validation loop
- 画曲线
- 可视化预测
- 解释代码

如果是分类任务，你的 ResNet transfer learning 模板和 MiniViT Transformer 模板都可以自然拆分到这些 cell 中。